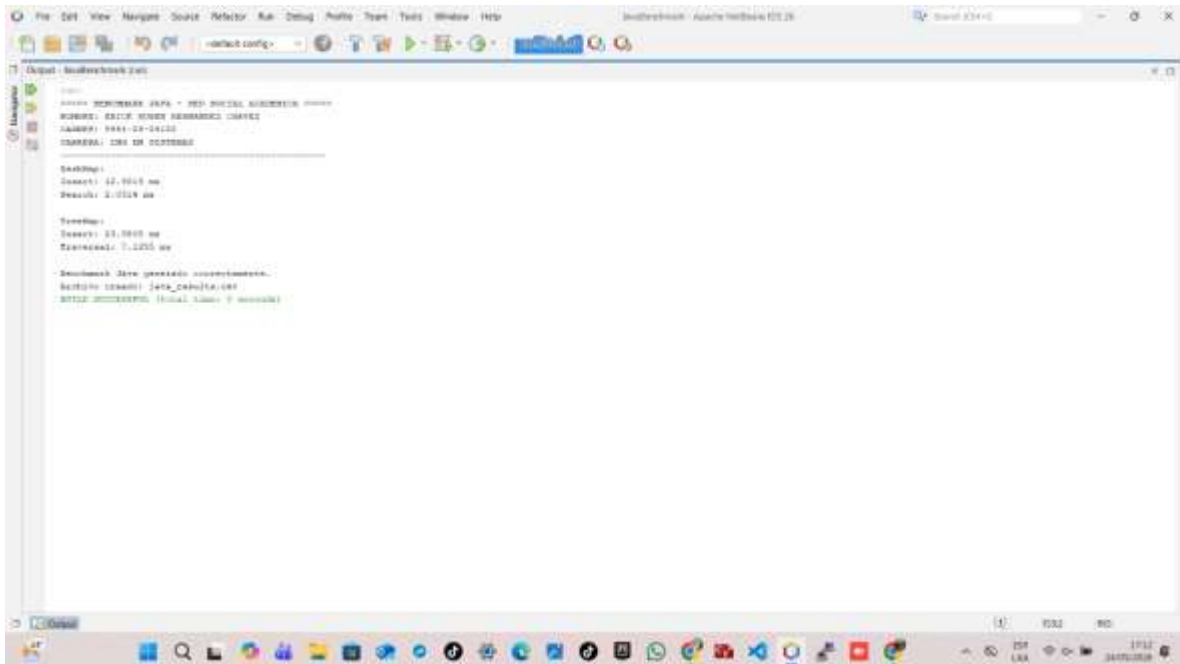
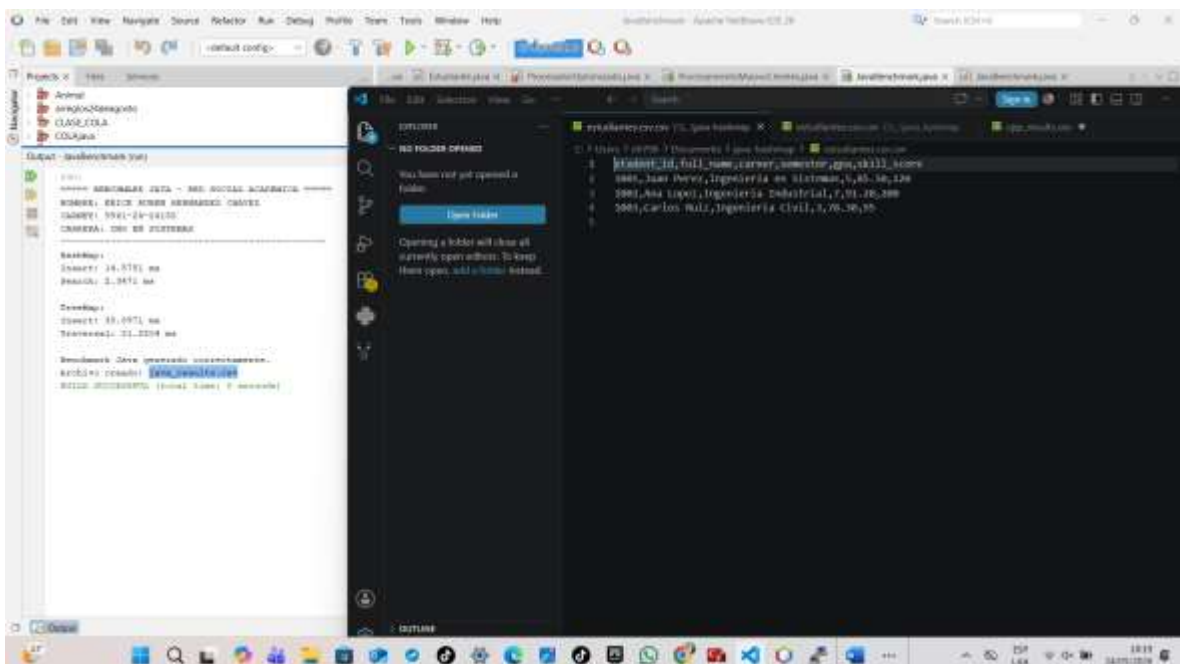


```
//EJECUCIÓN DE PROGRAMA JAVA //
```



```
// ARCHIVO CREADO POR JAVA//
java_results.csv
```



//CODIGO JAVA//

```
1 package javabenchmark;
2
3 import java.io.FileWriter;
4 import java.io.IOException;
5 import java.util.ArrayList;
6 import java.util.HashMap;
7 import java.util.Map;
8 import java.util.TreeMap;
9
10 public class JavaBenchmark {
11
12     static class Estudiante {
13         int studentId;
14         String fullName;
15         String career;
16         int semester;
17         double gpa;
18         int skillScore;
19
20         public Estudiante(int studentId, String fullName, String career, int semester, double gpa, int skillScore) {
21             this.studentId = studentId;
22             this.fullName = fullName;
23             this.career = career;
24             this.semester = semester;
25             this.gpa = gpa;
26             this.skillScore = skillScore;
27         }
28     }
29
30     static final int RECORDS = 10000;
31
32     public static void main(String[] args) {
33         mostrarEncabezado();
34         generarBenchmarkJava();
35     }
36
37     public static void mostrarEncabezado() {
```

```
37     public static void mostrarEncabezado() {
38         System.out.println("----- BENCHMARK JAVA - RED SOCIAL ACADEMICA -----");
39         System.out.println("NOMBRE: ERICK RUBEN HERNANDEZ CHAVES");
40         System.out.println("CARNET: 9941-24-24182");
41         System.out.println("CARRERA: ING EN SISTEMAS");
42         System.out.println("-----");
43     }
44
45     public static Estudiante crearEstudiante(int id) {
46         String nombre = "Estudiante " + id;
47         String carrera = "Ingenieria en Sistemas";
48         int semestre = (id % 10) + 1;
49         double gpa = 68.0 + (id % 41);
50         int skillScore = (id * 37) % 100000;
51
52         return new Estudiante(id, nombre, carrera, semestre, gpa, skillScore);
53     }
54
55     public static void generarBenchmarkJava() {
56         try {
57             FileWriter writer = new FileWriter("java_results.csv");
58
59             writer.write("language,operation,structure,records,time_ms\n");
60
61             benchmarkNashMap(writer);
62             benchmarkTreeMap(writer);
63
64             writer.close();
65
66             System.out.println();
67             System.out.println("Benchmark Java generado correctamente.");
68             System.out.println("Archivo creado: java_results.csv");
69
70         } catch (IOException e) {
71             System.out.println("Error al generar java_results.csv");
72             System.out.println(e.getMessage());
73         }
```

```

72         System.out.println(e.getMessage());
73     }
74 }
75
76 public static void benchmarkHashMap(FileWriter writer) throws IOException {
77     HashMap<Integer, Estudiante> estudiantesHash = new HashMap<>();
78
79     long inicioInsert = System.nanoTime();
80
81     for (int i = 1; i <= RECORDS; i++) {
82         Estudiante estudiante = crearEstudiante(i);
83         estudiantesHash.put(estudiante.studentId, estudiante);
84     }
85
86     long finInsert = System.nanoTime();
87
88     double tiempoInsert = (finInsert - inicioInsert) / 1_000_000.0;
89
90     writer.write("Java,insert,HashMap," + RECORDS + "," + tiempoInsert + "\n");
91
92     long inicioSearch = System.nanoTime();
93
94     for (int i = 1; i <= RECORDS; i++) {
95         estudiantesHash.get(i);
96     }
97
98     long finSearch = System.nanoTime();
99
100    double tiempoSearch = (finSearch - inicioSearch) / 1_000_000.0;
101
102    writer.write("Java,search,HashMap," + RECORDS + "," + tiempoSearch + "\n");
103
104    System.out.println("HashMap:");
105    System.out.println("Insert: " + tiempoInsert + " ms");
106    System.out.println("Search: " + tiempoSearch + " ms");
107 }
108

```

(4) 1553

```

109
110 public static void benchmarkTreeMap(FileWriter writer) throws IOException {
111     TreeMap<Integer, ArrayList<Estudiante>> estudiantesTree = new TreeMap<>();
112
113     long inicioInsert = System.nanoTime();
114
115     for (int i = 1; i <= RECORDS; i++) {
116         Estudiante estudiante = crearEstudiante(i);
117         int score = estudiante.skillScore;
118
119         if (!estudiantesTree.containsKey(score)) {
120             estudiantesTree.put(score, new ArrayList<>());
121         }
122
123         estudiantesTree.get(score).add(estudiante);
124     }
125
126     long finInsert = System.nanoTime();
127
128     double tiempoInsert = (finInsert - inicioInsert) / 1_000_000.0;
129
130     writer.write("Java,insert,TreeMap," + RECORDS + "," + tiempoInsert + "\n");
131
132     long inicioTraversal = System.nanoTime();
133
134     for (Map.Entry<Integer, ArrayList<Estudiante>> entrada : estudiantesTree.entrySet()) {
135         ArrayList<Estudiante> lista = entrada.getValue();
136
137         for (Estudiante estudiante : lista) {
138             int id = estudiante.studentId;
139             int score = estudiante.skillScore;
140         }
141     }
142
143     long finTraversal = System.nanoTime();
144
145     double tiempoTraversal = (finTraversal - inicioTraversal) / 1_000_000.0;
146

```

(4) 1553

```
..va Estudiante.java x ProcesadorOptimizado.java x ProcesamientoMasivoClientes.java x JavaBenchmark.java x JavaBenchmark.java x
Source History
118         if (!estudiantesTree.containsKey(score)) {
119             estudiantesTree.put(score, new ArrayList<>());
120         }
121
122         estudiantesTree.get(score).add(estudiante);
123     }
124
125     long finInsert = System.nanoTime();
126
127     double tiempoInsert = (finInsert - inicioInsert) / 1_000_000.0;
128
129     writer.write("Java,insert,TreeMap," + RECORDS + "," + tiempoInsert + "\n");
130
131     long inicioTraversal = System.nanoTime();
132
133     for (Map.Entry<Integer, ArrayList<Estudiante>> entrada : estudiantesTree.entrySet()) {
134         ArrayList<Estudiante> lista = entrada.getValue();
135
136         for (Estudiante estudiante : lista) {
137             int id = estudiante.studentId;
138             int score = estudiante.skillScore;
139         }
140     }
141
142     long finTraversal = System.nanoTime();
143
144     double tiempoTraversal = (finTraversal - inicioTraversal) / 1_000_000.0;
145
146     writer.write("Java,Traversal,TreeMap," + RECORDS + "," + tiempoTraversal + "\n");
147
148     System.out.println();
149     System.out.println("TreeMap:");
150     System.out.println("Insert: " + tiempoInsert + " ms");
151     System.out.println("Traversal: " + tiempoTraversal + " ms");
152 }
153
```